EXPLORATORY DATA ANALYSIS — CUSTOMER CHURN DATASET

ALGORITHM

1. Start

Begin the Customer Churn Exploratory Data Analysis program.

2. Import Libraries

Import NumPy, Pandas, Matplotlib, and Matplotlib Pyplot for data generation, data handling, and visualization.

3. Generate Synthetic Dataset

Create a synthetic customer churn dataset of 1000 records using a fixed random seed. Generate customer details such as gender, senior-citizen status, partner, dependents, tenure, internet service, contract, paperless billing, payment method, monthly charges, total charges, and churn.

4. Introduce Data Messiness

Deliberately introduce missing values, inconsistent capitalization, blank strings in TotalCharges, and duplicate rows to simulate common real-world data problems.

5. Explore Raw Data

Display the shape of the dataset, column data types, missing values, duplicate rows, and numerical summary statistics.

6. Clean the Data

Convert TotalCharges to numeric values, replace invalid blank values with NaN, remove extra whitespace, standardize Yes/No values, remove duplicate rows, fill missing tenure values with the median, calculate missing TotalCharges using MonthlyCharges × tenure, replace missing InternetService values with 'No', and reset the index.

7. Analyze Churn Patterns

Calculate the overall churn rate. Find churn rates by contract type and internet service. Compare average tenure and average monthly charges for churned and retained customers.

8. Create Visualizations

Create four charts: overall churn distribution, churn rate by contract type, tenure distribution by churn status, and a correlation heatmap for numeric features.

9. Save Results

Save the raw dataset as customer_churn_raw.csv, the cleaned dataset as customer_churn_cleaned.csv, and save all four charts as PNG files.

10. Display Completion Message

Display that EDA is complete and that the cleaned dataset is ready for further analysis.

11. Stop

End the program.

PYTHON CODE

The following is the uploaded program,reproducedwithoutchanging its logic.

```
"""
======================================================================
 EXPLORATORY DATA ANALYSIS \u2014 CUSTOMER CHURN DATASET
 Goal: explore the data, identify significant patterns behind churn,

       and clean the dataset so it's ready for further analysis
       (e.g. building a predictive model).
======================================================================
IN PLAIN ENGLISH:
 "Churn" means a customer leaving / cancelling a service. Before we
can build anything on top of this data (a report, a dashboard, a
prediction model), we first need to LOOK at it carefully: what does
it contain, what's messy about it, and what patterns separate
customers who leave from customers who stay?

 This script generates a realistic (synthetic) customer churn
 dataset \u2014 deliberately including the kinds of messiness real data
always has \u2014 then explores it, cleans it, analyzes it for
patterns, and saves both a cleaned CSV and a set of charts.
======================================================================
"""

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")          # render charts to files, no GUI needed
import matplotlib.pyplot as plt


# ======================================================================
# 1. GENERATE A REALISTIC (SYNTHETIC) CUSTOMER CHURN DATASET
#    Plain English: real datasets are never perfectly clean, so we
#    deliberately inject the kinds of problems real data has \u2014
#    missing values, duplicate rows, and inconsistent text formatting
#    \u2014 to give the cleaning step something real to fix.
# ======================================================================
def generate_synthetic_churn_data(n=1000, seed=42):
    """Returns a deliberately messy DataFrame shaped like a typical
    telecom customer-churn dataset."""
    rng = np.random.default_rng(seed)

    genders = rng.choice(["Male", "Female"], size=n)
    senior = rng.choice([0, 1], size=n, p=[0.84, 0.16])
    partner = rng.choice(["Yes", "No"], size=n)
    dependents = rng.choice(["Yes", "No"], size=n, p=[0.3, 0.7])
    tenure = rng.integers(0, 73, size=n).astype(float)          # months
    contract = rng.choice(
        ["Month-to-month", "One year", "Two year"], size=n, p=[0.55, 0.24, 0.21]
    )
    internet = rng.choice(["DSL", "Fiber optic", "No"], size=n, p=[0.35, 0.45, 0.2])
    paperless = rng.choice(["Yes", "No"], size=n, p=[0.6, 0.4])
    payment = rng.choice(
        ["Electronic check", "Mailed check", "Bank transfer", "Credit card"], size=n
    )
    monthly_charges = np.round(rng.normal(65, 30, size=n).clip(18, 120), 2)
    total_charges = np.round(monthly_charges * tenure + rng.normal(0, 50, size=n), 2)
    total_charges = np.clip(total_charges, 0, None)

         #----ChurnisNOTrandom: it depends on contract, tenure & charges,
    #          sothe"patterns"we look for later are genuinely there. ----
    churn_score = (
        (contract=="Month-to-month") * 0.35
        + (tenure < 12) * 0.30
        +(internet=="Fiberoptic") * 0.15
        +(monthly_charges>80) * 0.15
        +rng.normal(0,0.15,size=n)
    )
    churn=np.where(churn_score   > 0.45, "Yes", "No")

    df=pd.DataFrame({
        "customerID": [f"CUST-{10000+i}" for i in range(n)],
        "gender": genders,
        "SeniorCitizen": senior,
        "Partner": partner,
        "Dependents": dependents,
        "tenure": tenure,
```

```
        "InternetService": internet,
        "Contract": contract,
        "PaperlessBilling": paperless,
        "PaymentMethod": payment,
        "MonthlyCharges": monthly_charges,
        "TotalCharges": total_charges,
        "Churn": churn,
    })

    #----deliberately introduce realistic messiness ----------------------
    #1.Missing values in a few columns
    forcol,frac in [("TotalCharges", 0.03), ("tenure", 0.02), ("InternetService", 0.01)]:
        missing_idx = rng.choice(df.index, size=int(n * frac), replace=False)
        df.loc[missing_idx, col] = np.nan

    #2.Inconsistent capitalization in a text column (very common in
    #    real-world exports from different systems)
    messy_idx = rng.choice(df.index, size=int(n * 0.08), replace=False)
    df.loc[messy_idx, "Partner"] = df.loc[messy_idx, "Partner"].str.lower()

    #3.TotalCharges stored as text with stray blank strings (a classic
    #eal-world CSV problem \u2014 forces dtype to become 'object')
    blank_idx = rng.choice(df.index, size=int(n * 0.01), replace=False)
    df["TotalCharges"] = df["TotalCharges"].astype(object)
    df.loc[blank_idx, "TotalCharges"] = " "

    #4.Ahandful of exact duplicate rows
    dup_rows=df.sample(n=15, random_state=seed)
    df=pd.concat([df, dup_rows], ignore_index=True)

    return df


# ======================================================================
# 2. EXPLORE THE RAW DATA
#lain English: before touching anything, just LOOK \u2014 how big is
#t, what types are the columns, what's missing, what's duplicated?
# ======================================================================
def explore_raw_data(df):
    print("=" * 60)
    print("STEP 1: INITIAL EXPLORATION")
    print("=" * 60)
    print(f"Shape (rows, columns): {df.shape}")
    print(f"\nColumn data types:\n{df.dtypes}")
    print(f"\nMissing values per column:\n{df.isnull().sum()[df.isnull().sum() > 0]}")
    print(f"\nDuplicate rows: {df.duplicated().sum()}")
    print(f"\nNumeric summary:\n{df.select_dtypes(include=np.number).describe()}")


# ======================================================================
# 3. CLEAN THE DATA
#    Plain English: fix exactly the problems we found above, one at a
#    time, so the cleaned data is safe to analyze further.
# ======================================================================
def clean_data(df):
    """Returns a cleaned copy of df: fixed types, no duplicates, no
    inconsistent text, sensible handling of missing values."""
    clean = df.copy()

    #---- fix TotalCharges: blank strings -> NaN, then convert to float ---
    clean["TotalCharges"] = pd.to_numeric(clean["TotalCharges"], errors="coerce")

    #---- standardize text columns: trim whitespace everywhere ------------
    text_cols = clean.select_dtypes(include=["object", "str"]).columns.drop("customerID")
    forcol in text_cols:
        clean[col] = clean[col].astype(str).str.strip()

    #----standardize Yes/No columns specifically (case-insensitive) ------
    #      e.g. "yes", " Yes", "YES" all become the single consistent "Yes"
    yes_no_cols = ["Partner", "Dependents", "PaperlessBilling", "Churn"]
    forcolin yes_no_cols:
        clean[col] = clean[col].str.lower().map({"yes": "Yes", "no": "No"})

    #---- remove exact duplicate rows -------------------------------------
    before = len(clean)
    clean = clean.drop_duplicates()
    removed = before - len(clean)

    #---- handle missing values -------------------------------------------
    clean["tenure"] = clean["tenure"].fillna(clean["tenure"].median())
    clean["TotalCharges"] = clean["TotalCharges"].fillna(
        clean["MonthlyCharges"] * clean["tenure"]
```

```
    )
    clean["InternetService"] = clean["InternetService"].fillna("No")

    #---- reset a clean, contiguous index ------------------------------------
    clean = clean.reset_index(drop=True)

    print("=" * 60)
    print("STEP 2: DATA CLEANING SUMMARY")
    print("=" * 60)
    print(f"Duplicate rows removed : {removed}")
    print(f"Rows before cleaning   : {before}")
    print(f"Rows after cleaning    : {len(clean)}")
    print(f"Remaining missing values: {clean.isnull().sum().sum()}")
    return clean


# ======================================================================
# 4. ANALYZE PATTERNS BEHIND CHURN
#    Plain English: now that the data is trustworthy, look for
#    relationships between churn and everything else.
# ======================================================================
def analyze_churn_patterns(df):
    print("=" * 60)
    print("STEP 3: CHURN PATTERN ANALYSIS")
    print("=" * 60)

    overall_rate = (df["Churn"] == "Yes").mean() * 100
    print(f"Overall churn rate: {overall_rate:.1f}%\n")

    print("Churn rate by Contract type:")
    print((df.groupby("Contract")["Churn"].apply(lambda s: (s == "Yes").mean() * 100)
           .round(1).sort_values(ascending=False)))

    print("\nChurn rate by Internet Service:")
    print((df.groupby("InternetService")["Churn"].apply(lambda s: (s == "Yes").mean() * 100)
           .round(1).sort_values(ascending=False)))

    print("\nAverage tenure: churned vs. retained customers")
    print(df.groupby("Churn")["tenure"].mean().round(1))

    print("\nAverage monthly charges: churned vs. retained customers")
    print(df.groupby("Churn")["MonthlyCharges"].mean().round(2))


# ======================================================================
# 5. VISUALIZE THE FINDINGS
#    Plain English: a well-chosen chart reveals a pattern in one
#    glance that a table of numbers might hide.
# ======================================================================
def create_visualizations(df, output_dir="."):
plt.rcParams.update({"font.size": 11})

    #----Chart1:overallchurn distribution --------------------------
    fig,ax=plt.subplots(figsize=(5, 4))
    counts=df["Churn"].value_counts()
    ax.bar(counts.index,counts.values, color=["#0F766E", "#F97316"])
    ax.set_title("OverallChurn Distribution")
    ax.set_ylabel("NumberofCustomers")
    fori,vin    enumerate(counts.values):
        ax.text(i,v+5,str(v), ha="center", fontweight="bold")
    fig.tight_layout()
    fig.savefig(f"{output_dir}/chart_churn_distribution.png", dpi=150)
    plt.close(fig)

    #----Chart2:churnrate by contract type --------------------------
    fig,ax=plt.subplots(figsize=(6, 4))
    rates=df.groupby("Contract")["Churn"].apply(lambda s: (s == "Yes").mean() * 100)
    rates=rates.reindex(["Month-to-month", "One year", "Two year"])
    ax.bar(rates.index,rates.values, color="#0F766E")
    ax.set_title("ChurnRate by Contract Type")
    ax.set_ylabel("ChurnRate (%)")
    fori,vin    enumerate(rates.values):
        ax.text(i,v+1,f"{v:.1f}%", ha="center", fontweight="bold")
    fig.tight_layout()
    fig.savefig(f"{output_dir}/chart_churn_by_contract.png", dpi=150)
    plt.close(fig)

    #----Chart3: tenure distribution by churn ----------------------------
    fig,ax=plt.subplots(figsize=(6, 4))
    forlabel,color in [("No", "#0F766E"), ("Yes", "#F97316")]:
        subset=df[df["Churn"] == label]["tenure"]
        ax.hist(subset, bins=20, alpha=0.6, label=f"Churn = {label}", color=color)
```

```
    ax.set_title("Tenure Distribution by Churn Status")
    ax.set_xlabel("Tenure (months)")
    ax.set_ylabel("Number of Customers")
    ax.legend()
    fig.tight_layout()
    fig.savefig(f"{output_dir}/chart_tenure_by_churn.png", dpi=150)
    plt.close(fig)

    #----Chart4:  correlation  heatmap  of  numeric  features  -------------------
    fig,ax=plt.subplots(figsize=(5, 4.5))
    numeric_df=df[["SeniorCitizen",  "tenure",  "MonthlyCharges",  "TotalCharges"]]
    corr = numeric_df.corr()
    im=ax.imshow(corr,           cmap="RdYlGn",           vmin=-1,           vmax=1)
    ax.set_xticks(range(len(corr.columns))) ax.set_yticks(range(len(corr.columns)))
    ax.set_xticklabels(corr.columns,            rotation=45,            ha="right")
    ax.set_yticklabels(corr.columns)
    for i in range(len(corr)):

        forjinrange(len(corr)):
            ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center", fontsize=9)
    ax.set_title("Correlation Between Numeric Features")
    fig.colorbar(im, ax=ax, shrink=0.8)
    fig.tight_layout()
    fig.savefig(f"{output_dir}/chart_correlation_heatmap.png", dpi=150)
    plt.close(fig)

    print("=" * 60)
    print("STEP 4: VISUALIZATIONS SAVED")
    print("=" * 60)
    print(" chart_churn_distribution.png")
    print(" chart_churn_by_contract.png")
    print(" chart_tenure_by_churn.png")
    print(" chart_correlation_heatmap.png")


# ======================================================================
# 6. DEMO / DRIVER CODE
# ======================================================================
if __name__ == "__main__":

    #---- Step 0: generate (or, in a real project, load) the data --------
    raw_df = generate_synthetic_churn_data(n=1000)
    raw_df.to_csv("customer_churn_raw.csv", index=False)
    print(f"Raw dataset saved: customer_churn_raw.csv ({raw_df.shape[0]} rows)\n")

    #---- Step 1: explore it before touching anything ----------------------
    explore_raw_data(raw_df)

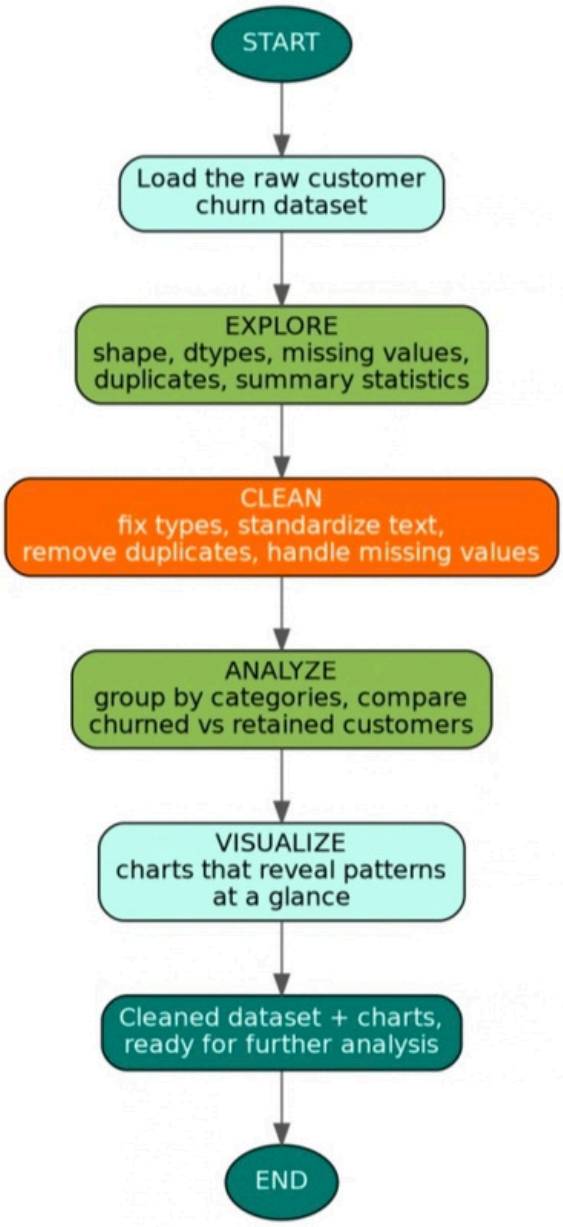
    #---- Step 2: clean it -------------------------------------------------
    clean_df = clean_data(raw_df)
    clean_df.to_csv("customer_churn_cleaned.csv", index=False)
    print(f"\nCleaned dataset saved: customer_churn_cleaned.csv ({clean_df.shape[0]} rows)")

    #---- Step 3: analyze patterns behind churn ----------------------------
    analyze_churn_patterns(clean_df)

    #---- Step 4: visualize the findings -----------------------------------
    create_visualizations(clean_df)

    print("\nEDA complete. The cleaned dataset is ready for further analysis.")
```

FLOWCHART


START
Load the raw customer churn dataset
EXPLORE
shape, dtypes, missing values, duplicates, summary statistics
CLEAN
fix types, standardize text, remove duplicates, handle missing values
ANALYZE
group by categories, compare churned vs retained customers
VISUALIZE
charts that reveal patterns at a glance
Cleaned dataset + charts, ready for further analysis
END

main.py

```
"""
=====================================================================
 EXPLORATORY DATA ANALYSIS \u2014 CUSTOMER CHURN DATASET
 Goal: explore the data, identify significant patterns behind churn,
       and clean the dataset so it's ready for further analysis
       (e.g. building a predictive model).
=====================================================================
IN PLAIN ENGLISH:
  "Churn" means a customer leaving / cancelling a service. Before we
  can build anything on top of this data (a report, a dashboard, a
  prediction model), we first need to LOOK at it carefully: what does
  it contain, what's messy about it, and what patterns separate
  customers who leave from customers who stay?

  This script generates a realistic (synthetic) customer churn
  dataset \u2014 deliberately including the kinds of messiness real data
  always has \u2014 then explores it, cleans it, analyzes it for
  patterns, and saves both a cleaned CSV and a set of charts.
=====================================================================
"""

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")          # render charts to files, no GUI needed
import matplotlib.pyplot as plt


# =====================================================================
# 1. GENERATE A REALISTIC (SYNTHETIC) CUSTOMER CHURN DATASET
#    Plain English: real datasets are never perfectly clean, so we
#    deliberately inject the kinds of problems real data has \u2014
#    missing values, duplicate rows, and inconsistent text formatting
```

Ready Terminal

29°C Sunny

Search

main.py

```
import matplotlib.pyplot as plt


# =====================================================================
# 1. GENERATE A REALISTIC (SYNTHETIC) CUSTOMER CHURN DATASET
#    Plain English: real datasets are never perfectly clean, so we
#    deliberately inject the kinds of problems real data has \u2014
#    missing values, duplicate rows, and inconsistent text formatting
#    \u2014 to give the cleaning step something real to fix.
# =====================================================================
def generate_synthetic_churn_data(n=1000, seed=42):
    """Returns a deliberately messy DataFrame shaped like a typical
    telecom customer-churn dataset."""
    rng = np.random.default_rng(seed)

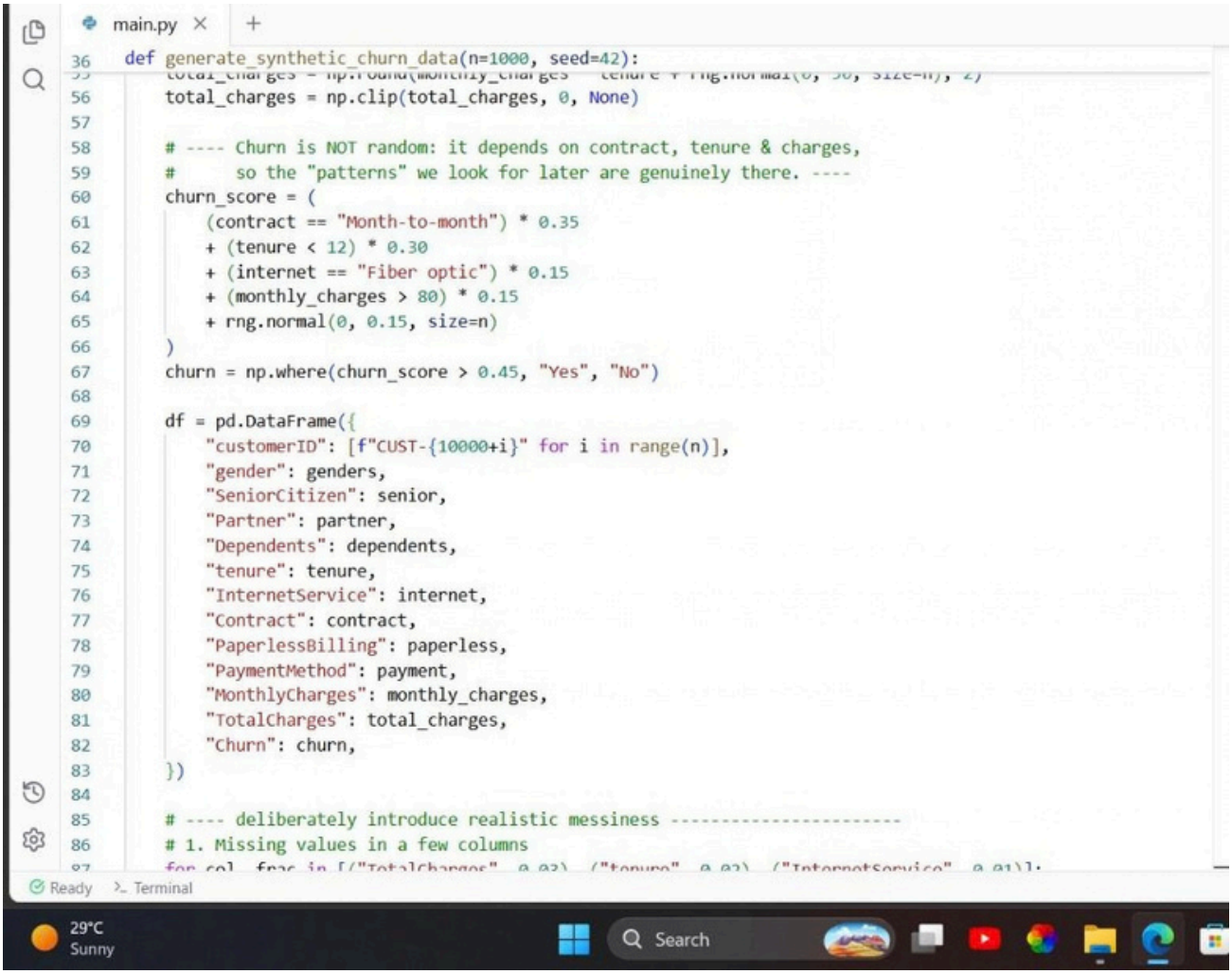
    genders = rng.choice(["Male", "Female"], size=n)
    senior = rng.choice([0, 1], size=n, p=[0.84, 0.16])
    partner = rng.choice(["Yes", "No"], size=n)
    dependents = rng.choice(["Yes", "No"], size=n, p=[0.3, 0.7])
    tenure = rng.integers(0, 73, size=n).astype(float)          # months
    contract = rng.choice(
        ["Month-to-month", "One year", "Two year"], size=n, p=[0.55, 0.24, 0.21]
    )
    internet = rng.choice(["DSL", "Fiber optic", "No"], size=n, p=[0.35, 0.45, 0.2])
    paperless = rng.choice(["Yes", "No"], size=n, p=[0.6, 0.4])
    payment = rng.choice(
        ["Electronic check", "Mailed check", "Bank transfer", "Credit card"], size=n
    )
    monthly_charges = np.round(rng.normal(65, 30, size=n).clip(18, 120), 2)
    total_charges = np.round(monthly_charges * tenure + rng.normal(0, 50, size=n), 2)
    total_charges = np.clip(total_charges, 0, None)

    # ---- Churn is NOT random: it depends on contract, tenure & charges,
```

Ready Terminal

29°C Sunny

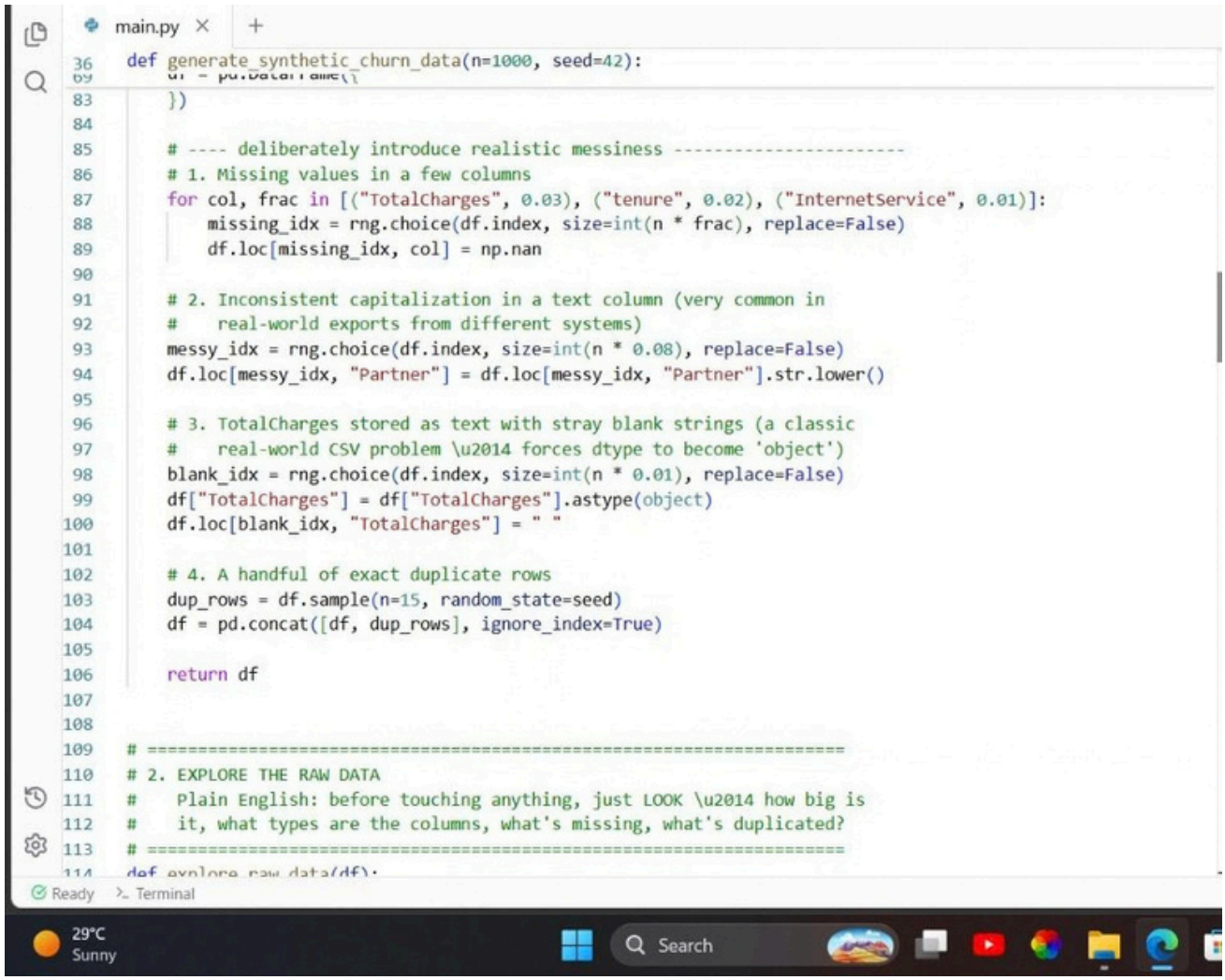
Search



```
main.py
36  def generate_synthetic_churn_data(n=1000, seed=42):
56      total_charges = np.clip(total_charges, 0, None)
57
58      # ---- Churn is NOT random: it depends on contract, tenure & charges,
59      #      so the "patterns" we look for later are genuinely there. ----
60      churn_score = (
61          (contract == "Month-to-month") * 0.35
62          + (tenure < 12) * 0.30
63          + (internet == "Fiber optic") * 0.15
64          + (monthly_charges > 80) * 0.15
65          + rng.normal(0, 0.15, size=n)
66      )
67      churn = np.where(churn_score > 0.45, "Yes", "No")
68
69      df = pd.DataFrame({
70          "customerID": [f"CUST-{10000+i}" for i in range(n)],
71          "gender": genders,
72          "SeniorCitizen": senior,
73          "Partner": partner,
74          "Dependents": dependents,
75          "tenure": tenure,
76          "InternetService": internet,
77          "Contract": contract,
78          "PaperlessBilling": paperless,
79          "PaymentMethod": payment,
80          "MonthlyCharges": monthly_charges,
81          "TotalCharges": total_charges,
82          "Churn": churn,
83      })
84
85      # ---- deliberately introduce realistic messiness ----------------------
86      # 1. Missing values in a few columns
```

Ready Terminal

29°C Sunny Search



```
main.py
36  def generate_synthetic_churn_data(n=1000, seed=42):
83      })
84
85      # ---- deliberately introduce realistic messiness ----------------------
86      # 1. Missing values in a few columns
87      for col, frac in [("TotalCharges", 0.03), ("tenure", 0.02), ("InternetService", 0.01)]:
88          missing_idx = rng.choice(df.index, size=int(n * frac), replace=False)
89          df.loc[missing_idx, col] = np.nan
90
91      # 2. Inconsistent capitalization in a text column (very common in
92      #    real-world exports from different systems)
93      messy_idx = rng.choice(df.index, size=int(n * 0.08), replace=False)
94      df.loc[messy_idx, "Partner"] = df.loc[messy_idx, "Partner"].str.lower()
95
96      # 3. TotalCharges stored as text with stray blank strings (a classic
97      #    real-world CSV problem \u2014 forces dtype to become 'object')
98      blank_idx = rng.choice(df.index, size=int(n * 0.01), replace=False)
99      df["TotalCharges"] = df["TotalCharges"].astype(object)
100     df.loc[blank_idx, "TotalCharges"] = " "
101
102     # 4. A handful of exact duplicate rows
103     dup_rows = df.sample(n=15, random_state=seed)
104     df = pd.concat([df, dup_rows], ignore_index=True)
105
106     return df
107
108
109 # ======================================================================
110 # 2. EXPLORE THE RAW DATA
111 #    Plain English: before touching anything, just LOOK \u2014 how big is
112 #    it, what types are the columns, what's missing, what's duplicated?
113 # ======================================================================
```

Ready Terminal

29°C Sunny Search

main.py

```
#    it, what types are the columns, what's missing, what's duplicated?
# ====================================================================
def explore_raw_data(df):
    print("=" * 60)
    print("STEP 1: INITIAL EXPLORATION")
    print("=" * 60)
    print(f"Shape (rows, columns): {df.shape}")
    print(f"\nColumn data types:\n{df.dtypes}")
    print(f"\nMissing values per column:\n{df.isnull().sum()[df.isnull().sum() > 0]}")
    print(f"\nDuplicate rows: {df.duplicated().sum()}")
    print(f"\nNumeric summary:\n{df.select_dtypes(include=np.number).describe()}")


# ====================================================================
# 3. CLEAN THE DATA
#    Plain English: fix exactly the problems we found above, one at a
#    time, so the cleaned data is safe to analyze further.
# ====================================================================
def clean_data(df):
    """Returns a cleaned copy of df: fixed types, no duplicates, no
    inconsistent text, sensible handling of missing values."""
    clean = df.copy()

    # ---- fix TotalCharges: blank strings -> NaN, then convert to float ---
    clean["TotalCharges"] = pd.to_numeric(clean["TotalCharges"], errors="coerce")

    # ---- standardize text columns: trim whitespace everywhere ------------
    text_cols = clean.select_dtypes(include=["object", "str"]).columns.drop("customerID")
    for col in text_cols:
        clean[col] = clean[col].astype(str).str.strip()

    # ---- standardize Yes/No columns specifically (case-insensitive) ------
    #      e.g. "yes", " Yes", "YES" all become the single consistent "Yes"
```

Ready Terminal

29°C Sunny

Search

main.py

```
def clean_data(df):
    # ---- standardize Yes/No columns specifically (case-insensitive) ------
    #      e.g. "yes", " Yes", "YES" all become the single consistent "Yes"
    yes_no_cols = ["Partner", "Dependents", "PaperlessBilling", "Churn"]
    for col in yes_no_cols:
        clean[col] = clean[col].str.lower().map({"yes": "Yes", "no": "No"})

    # ---- remove exact duplicate rows -------------------------------------
    before = len(clean)
    clean = clean.drop_duplicates()
    removed = before - len(clean)

    # ---- handle missing values -------------------------------------------
    clean["tenure"] = clean["tenure"].fillna(clean["tenure"].median())
    clean["TotalCharges"] = clean["TotalCharges"].fillna(
        clean["MonthlyCharges"] * clean["tenure"]
    )
    clean["InternetService"] = clean["InternetService"].fillna("No")

    # ---- reset a clean, contiguous index ---------------------------------
    clean = clean.reset_index(drop=True)

    print("=" * 60)
    print("STEP 2: DATA CLEANING SUMMARY")
    print("=" * 60)
    print(f"Duplicate rows removed : {removed}")
    print(f"Rows before cleaning   : {before}")
    print(f"Rows after cleaning    : {len(clean)}")
    print(f"Remaining missing values: {clean.isnull().sum().sum()}")
    return clean


# ---------------------------------------------------------------------
```

Ready Terminal

29°C Sunny

Search

main.py

```
def clean_data(df):
    return clean


# ======================================================================
# 4. ANALYZE PATTERNS BEHIND CHURN
#    Plain English: now that the data is trustworthy, look for
#    relationships between churn and everything else.
# ======================================================================
def analyze_churn_patterns(df):
    print("=" * 60)
    print("STEP 3: CHURN PATTERN ANALYSIS")
    print("=" * 60)

    overall_rate = (df["Churn"] == "Yes").mean() * 100
    print(f"Overall churn rate: {overall_rate:.1f}%\n")

    print("Churn rate by Contract type:")
    print((df.groupby("Contract")["Churn"].apply(lambda s: (s == "Yes").mean() * 100)
           .round(1).sort_values(ascending=False)))

    print("\nChurn rate by Internet Service:")
    print((df.groupby("InternetService")["Churn"].apply(lambda s: (s == "Yes").mean() * 100)
           .round(1).sort_values(ascending=False)))

    print("\nAverage tenure: churned vs. retained customers")
    print(df.groupby("Churn")["tenure"].mean().round(1))

    print("\nAverage monthly charges: churned vs. retained customers")
    print(df.groupby("Churn")["MonthlyCharges"].mean().round(2))


# ======================================================================
```

Ready Terminal

29°C Sunny Search

main.py

```


# ======================================================================
# 5. VISUALIZE THE FINDINGS
#    Plain English: a well-chosen chart reveals a pattern in one
#    glance that a table of numbers might hide.
# ======================================================================
def create_visualizations(df, output_dir="."):
    plt.rcParams.update({"font.size": 11})

    # ---- Chart 1: overall churn distribution -------------------------
    fig, ax = plt.subplots(figsize=(5, 4))
    counts = df["Churn"].value_counts()
    ax.bar(counts.index, counts.values, color=["#0F766E", "#F97316"])
    ax.set_title("Overall Churn Distribution")
    ax.set_ylabel("Number of Customers")
    for i, v in enumerate(counts.values):
        ax.text(i, v + 5, str(v), ha="center", fontweight="bold")
    fig.tight_layout()
    fig.savefig(f"{output_dir}/chart_churn_distribution.png", dpi=150)
    plt.close(fig)

    # ---- Chart 2: churn rate by contract type ------------------------
    fig, ax = plt.subplots(figsize=(6, 4))
    rates = df.groupby("Contract")["Churn"].apply(lambda s: (s == "Yes").mean() * 100)
    rates = rates.reindex(["Month-to-month", "One year", "Two year"])
    ax.bar(rates.index, rates.values, color="#0F766E")
    ax.set_title("Churn Rate by Contract Type")
    ax.set_ylabel("Churn Rate (%)")
    for i, v in enumerate(rates.values):
        ax.text(i, v + 1, f"{v:.1f}%", ha="center", fontweight="bold")
    fig.tight_layout()
    fig.savefig(f"{output_dir}/chart_churn_by_contract.png", dpi=150)
```

Ready Terminal

29°C Sunny Search

main.py

```
def create_visualizations(df, output_dir="."):
    for i, v in enumerate(rates.values):
    fig.tight_layout()
    fig.savefig(f"{output_dir}/chart_churn_by_contract.png", dpi=150)
    plt.close(fig)

    # ---- Chart 3: tenure distribution by churn -----------------------------
    fig, ax = plt.subplots(figsize=(6, 4))
    for label, color in [("No", "#0F766E"), ("Yes", "#F97316")]:
        subset = df[df["Churn"] == label]["tenure"]
        ax.hist(subset, bins=20, alpha=0.6, label=f"Churn = {label}", color=color)
    ax.set_title("Tenure Distribution by Churn Status")
    ax.set_xlabel("Tenure (months)")
    ax.set_ylabel("Number of Customers")
    ax.legend()
    fig.tight_layout()
    fig.savefig(f"{output_dir}/chart_tenure_by_churn.png", dpi=150)
    plt.close(fig)

    # ---- Chart 4: correlation heatmap of numeric features ------------------
    fig, ax = plt.subplots(figsize=(5, 4.5))
    numeric_df = df[["SeniorCitizen", "tenure", "MonthlyCharges", "TotalCharges"]]
    corr = numeric_df.corr()
    im = ax.imshow(corr, cmap="RdYlGn", vmin=-1, vmax=1)
    ax.set_xticks(range(len(corr.columns)))
    ax.set_yticks(range(len(corr.columns)))
    ax.set_xticklabels(corr.columns, rotation=45, ha="right")
    ax.set_yticklabels(corr.columns)
    for i in range(len(corr)):
        for j in range(len(corr)):
            ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center", fontsize=9)
    ax.set_title("Correlation Between Numeric Features")
    fig.colorbar(im, ax=ax, shrink=0.8)
```

Ready Terminal

29°C Sunny

Search

main.py

```
def create_visualizations(df, output_dir="."):
    ax.set_title("Correlation Between Numeric Features")
    fig.colorbar(im, ax=ax, shrink=0.8)
    fig.tight_layout()
    fig.savefig(f"{output_dir}/chart_correlation_heatmap.png", dpi=150)
    plt.close(fig)

    print("=" * 60)
    print("STEP 4: VISUALIZATIONS SAVED")
    print("=" * 60)
    print("  chart_churn_distribution.png")
    print("  chart_churn_by_contract.png")
    print("  chart_tenure_by_churn.png")
    print("  chart_correlation_heatmap.png")


# ======================================================================
# 6. DEMO / DRIVER CODE
# ======================================================================
if __name__ == "__main__":

    # ---- Step 0: generate (or, in a real project, load) the data --------
    raw_df = generate_synthetic_churn_data(n=1000)
    raw_df.to_csv("customer_churn_raw.csv", index=False)
    print(f"Raw dataset saved: customer_churn_raw.csv ({raw_df.shape[0]} rows)\n")

    # ---- Step 1: explore it before touching anything ---------------------
    explore_raw_data(raw_df)

    # ---- Step 2: clean it ------------------------------------------------
    clean_df = clean_data(raw_df)
    clean_df.to_csv("customer_churn_cleaned.csv", index=False)
    print(f"\ncleaned dataset saved: customer_churn_cleaned.csv ({clean_df.shape[0]} rows)")
```

Ready Terminal

29°C Sunny

Search

main.py

```
def create_visualizations(df, output_dir="."):
    print("  chart_churn_distribution.png")
    print("  chart_churn_by_contract.png")
    print("  chart_tenure_by_churn.png")
    print("  chart_correlation_heatmap.png")


# ======================================================================
# 6. DEMO / DRIVER CODE
# ======================================================================
if __name__ == "__main__":

    # ---- Step 0: generate (or, in a real project, load) the data --------
    raw_df = generate_synthetic_churn_data(n=1000)
    raw_df.to_csv("customer_churn_raw.csv", index=False)
    print(f"Raw dataset saved: customer_churn_raw.csv ({raw_df.shape[0]} rows)\n")

    # ---- Step 1: explore it before touching anything --------------------
    explore_raw_data(raw_df)

    # ---- Step 2: clean it -----------------------------------------------
    clean_df = clean_data(raw_df)
    clean_df.to_csv("customer_churn_cleaned.csv", index=False)
    print(f"\nCleaned dataset saved: customer_churn_cleaned.csv ({clean_df.shape[0]} rows)")

    # ---- Step 3: analyze patterns behind churn --------------------------
    analyze_churn_patterns(clean_df)

    # ---- Step 4: visualize the findings ---------------------------------
    create_visualizations(clean_df)

    print("\nEDA complete. The cleaned dataset is ready for further analysis.")

```

Ready Terminal

29°C Sunny Search

```
Raw dataset saved: customer_churn_raw.csv (1015 rows)

============================================================
STEP 1: INITIAL EXPLORATION
============================================================
Shape (rows, columns): (1015, 13)

Column data types:
customerID              str
gender                  str
SeniorCitizen         int64
Partner                 str
Dependents              str
tenure              float64
InternetService         str
Contract                str
PaperlessBilling        str
PaymentMethod           str
MonthlyCharges      float64
TotalCharges         object
Churn                   str
dtype: object

Missing values per column:
tenure            20
InternetService   11
TotalCharges      30
dtype: int64

Duplicate rows: 15
```

Light Wiki Internet

ENG IN 10:30 06-10-2026

```
dtype: int64

Duplicate rows: 15

Numeric summary:
       SeniorCitizen      tenure  MonthlyCharges
count    1015.000000  995.000000     1015.000000
mean        0.182266   34.102513       64.335222
std         0.386254   21.190226       26.951266
min         0.000000    0.000000       18.000000
25%         0.000000   16.000000       44.890000
50%         0.000000   34.000000       63.030000
75%         0.000000   52.000000       83.410000
max         1.000000   72.000000      120.000000
============================================================
STEP 2: DATA CLEANING SUMMARY
============================================================
Duplicate rows removed : 15
Rows before cleaning   : 1015
Rows after cleaning    : 1000
Remaining missing values: 0

Cleaned dataset saved: customer_churn_cleaned.csv (1000 rows)
============================================================
STEP 3: CHURN PATTERN ANALYSIS
============================================================
Overall churn rate: 37.9%

Churn rate by Contract type:
Contract
```

Light Wiki Internet

ENG IN 10:30 06-10-2026

```
STEP 3: CHURN PATTERN ANALYSIS
============================================================
Overall churn rate: 37.9%

Churn rate by Contract type:
Contract
Month-to-month    57.6
Two year          12.5
One year          10.8
Name: Churn, dtype: float64

Churn rate by Internet Service:
InternetService
Fiber optic    48.8
No             30.4
DSL            29.3
Name: Churn, dtype: float64

Average tenure: churned vs. retained customers
Churn
No     38.0
Yes    28.0
Name: tenure, dtype: float64

Average monthly charges: churned vs. retained customers
Churn
No     61.86
Yes    68.36
Name: MonthlyCharges, dtype: float64
============================================================
```

Light Wiki Internet

ENG IN 10:31 06-10-2026

```
One year          10.8
Name: Churn, dtype: float64

Churn rate by Internet Service:
InternetService
Fiber optic    48.8
No             30.4
DSL            29.3
Name: Churn, dtype: float64

Average tenure: churned vs. retained customers
Churn
No     38.0
Yes    28.0
Name: tenure, dtype: float64

Average monthly charges: churned vs. retained customers
Churn
No     61.86
Yes    68.36
Name: MonthlyCharges, dtype: float64
============================================================
STEP 4: VISUALIZATIONS SAVED
============================================================
  chart_churn_distribution.png
  chart_churn_by_contract.png
  chart_tenure_by_churn.png
  chart_correlation_heatmap.png

EDA complete. The cleaned dataset is ready for further analysis.
```

Light Wiki Internet

ENG IN 10:31 06-10-2026